



UNIVERSITY OF HERTFORDSHIRE

School of Physics, Engineering and Computer Science

MSc in Software Engineering

7COM1038 and Software Engineering Masters Project

Date: 27th April 2026

A Comparative Evaluation of Machine Learning
Detection and Behavioral Warning Interventions in
Reducing User Susceptibility to Phishing Attacks

Name: Md. Abidur Rahman Shafi

Student ID: 24079469

Supervisor: Dr. Kelechi Emerole

Declaration:

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science in Software Engineering, at the University of Hertfordshire (UH). I hereby declare that the work presented in this project and report is entirely my own, except where explicitly stated otherwise. All sources of information and ideas, whether quoted directly or paraphrased, have been properly referenced in accordance with academic standards. I understand that any failure to properly acknowledge the work of others could constitute plagiarism and may result in academic penalties.

Use of Secondary Data: This project makes use of secondary datasets, specifically the PhishTank verified phishing URL and the Tranco top-1 million domain ranking, both of which are publicly available, fully anonymized, and depersonalized datasets. Neither dataset contains personally identifiable information. The PhishTank dataset consists solely of URLs submitted and verified by the public community for the explicit purpose of phishing research and detection. The Tranco dataset consists of domain name rankings derived from aggregated web traffic measurements, containing no individual-level personal data.

In accordance with data protection principles, only the data strictly necessary for model training and evaluation was used. No unpublished or unanonymized secondary data was processed at any stage of this project. As both datasets are published, anonymized, and collected with appropriate permissions for research use, ethics approval was not required for their use in this study.

With respect to behavioral event data collected through Firebase Analytics during application deployment: no personally identifiable information was collected at any point. Firebase Analytics uses pseudonymous device-instance identifiers that cannot be reverse tracked to individual users. Users provided explicit informed consent via the application's consent page prior to any data collection occurring. I did not use human participants in the traditional sense of the term but behavioral data was collected passively through anonymized event logging with prior user consent. I hereby give permission for this report to be made available on the university website provided the source is acknowledged.

I confirm that this report has been critically proof-read and quality checked, ensuring it is free from grammar, spelling, and formatting errors, and meets high standards of clarity, coherence, and presentation.

Signed: Md Abidur Rahman Shafi

Date: 27 April 2026

Table of Contents

1. Abstract	3
2. Introduction	4
2.1 Problem Overview and Significance	4
2.2 Research Gap and Motivation	4
2.3 Research Question and Sub-Questions	5
2.4 Project Overview	5
2.5 Aims and Objectives	5
2.6 Feasibility, Ethics, and Commercial Context	6
2.7 Report Structure	6
3. Literature Review	7
3.1 Overview	7
3.2 Phishing: Mechanisms, Scale, and the Persistence Problem	7
3.3 Machine Learning for Phishing URL Detection	8
3.3.1 Feature Taxonomies and Lexical Approaches	8
3.3.2 Gradient-Boosted Models and LightGBM	8
3.3.3 The Deployment Gap	9
3.4 Usable Security: Warning Design and Human Decision-Making	9
3.4.1 The Effectiveness of Browser Security Warnings	9
3.4.2 Warning Fatigue and Habituation	9
3.4.3 Attention Attractors and Warning Design Principles	10
3.4.4 Behavioural Economics: System 1/2 and Prospect Theory	10
3.5 Community-Driven Detection	11
3.6 Synthesis and Gap Identification	11
4. Methodology	12
4.1 Research Design	12
4.2 Machine Learning Model: Design and Justification	12
4.2.1 Dataset Composition and Labelling	12
4.2.2 Protocol-Neutral Feature Engineering	13
4.2.3 Classifier Selection and Hyperparameter Justification	13
4.3 Backend API: Architecture and Deployment	14
4.4 Flutter Application: Architecture and Warning Screen Design	14
4.4.1 Application Structure and Technology Stack	14

4.4.2 The Link Click Interaction and Warning Screen	14
4.5 Data Collection: Event Schema and BigQuery Pipeline	15
4.6 Consent Gate and Ethical Considerations	16
4.7 Testing Strategy	16
5. Quality and Results	17
5.1 Machine Learning Model Performance	17
5.2 Behavioral Results: Link Interaction Overview	18
5.3 Warning Compliance and Session Abandonment	19
5.4 Decision Latency Analysis	20
5.5 Warning Type Distribution and Data Quality	21
5.6 Community Reporting System	21
5.7 Technical Performance	22
6. Evaluation and Conclusion	23
6.1 Evaluation Against Objectives	23
6.2 Critical Evaluation of the ML Model	23
6.3 Critical Evaluation of the Behavioral Results	23
6.4 Project Management Reflection	24
6.5 Comparison with Literature	25
6.6 Limitations	25
6.7 Future Work	26
6.8 Conclusion	26
7. References	27
Appendix A: System Architecture	28
Appendix B: BigQuery SQL Queries	28
Appendix C: Firebase Analytics Event Schema	29

1. Abstract

Phishing attacks, which exploit human weaknesses to get credentials or facilitate malware installation, continue to be one of the most economically significant cybersecurity risks worldwide. The success rate of automated detection systems in the real world depends on how users react when classifiers reveal threat signals, a factor that has gotten relatively little empirical research despite significant investment in these systems. By creating, implementing, and instrumenting PhishAware LinkBox, a community-driven link-sharing mobile application that serves as both a controlled behavioral study tool and a realistic social platform, this initiative directly fills that gap.

Using 22 protocol-neutral lexical features extracted from the URL string alone, without requiring page content retrieval, a LightGBM classifier was trained on a balanced dataset of 20,000 URLs, 10,000 of which were verified phishing samples from the PhishTank community-verified feed and 10,000 of which were legitimate domains from the Tranco top-1 million ranking. About 16,000 training samples and 4,000 test samples were produced using an 80/20 train-test split (random_state=42). On the held-out test set, the trained model obtained 99.4% accuracy, 99.6% precision, and a false positive rate of 0.4%.

The Flutter mobile application, backed by Firebase Firestore, presents community-shared links for browsing; each click triggers real-time ML inference and, where a phishing classification is returned, routes the user through an animated warning screen requiring an explicit decision.

Every user interaction was recorded in Firebase Analytics and exported to Google BigQuery, including link clicks, warning exposures, and explicit proceed or cancel decisions. 56 warning screen exposures were found by analyzing event data from 66 total link engagements. Of these, 18 resulted in a logged explicit decision (14 cancellations, 4 proceed-overrides), while 38 users ended the session completely without pressing either button. The combined ML-plus-warning intervention significantly impedes navigation to phishing URLs, as seen by the 77.8% cancellation rate among explicit decisions and the 67.9% total avoidance rate when session abandonments are taken into account.

The assumption that warning overrides reflect habitual dismissal is challenged by the finding that users who proceeded despite warnings spent a longer mean decision time (7,824ms) than those who cancelled (7,085ms). This finding suggests deliberate conscious override, which has significant implications for future security interface design. A small sample size, a user demographic that is probably more security-aware than average, and a null-logged parameter problem in 22 out of 56 warning events are among the limitations.

2. Introduction

2.1 Problem Overview and Significance

The sustained prevalence of phishing as a primary initial access vector in cybercrime represents one of the most consequential unsolved problems in applied security. The Anti-Phishing Working Group (APWG) recorded a record high of 1,624,144 phishing attacks in the first quarter of 2023 alone. Although Q1 2024 recorded a lower figure of 963,994, the overall year-over-year trend shows roughly 1.9 million attacks over the twelve-month period from May 2023 to April 2024.

Phishing was found to be the initial access vector in 15% of confirmed data breaches across 10,626 events examined in the Verizon Data Breach Investigations Report (2024). This percentage increases significantly when pre-texting and social engineering variants are taken into account. In 2023 alone, phishing-facilitated credential theft caused business email compromise damages in the US of USD 2.9 billion (FBI IC3, reported in APWG, 2024b).

These numbers are especially alarming because they continue to exist in spite of the extensive use of automated detection technologies. The majority of workplace email gateways use machine learning (ML)-based filtering; URL reputation systems keep up-to-date blacklists; and every major browser comes with phishing protection (Google Safe Browsing, Microsoft SmartScreen). It is implied that detection without efficient behavioral mediation is inadequate rather than that automated detection has failed.

A user who ignores the resulting warning cannot be protected by a classifier that correctly detects a phishing URL. The gap between detection accuracy and real user protection, which exists at the intersection of machine learning and usable security, is examined in this study.

2.2 Research Gap and Motivation

This work is informed by two research traditions that are essentially parallel. The first, which focuses on machine learning and security, has created URL-based phishing classifiers that are getting more and more accurate. Gradient-boosted models, such as LightGBM, regularly outperform previous methods, according to comparative studies. Lexical URL characteristics offer real-time inference without requiring page content retrieval, which is a crucial operational advantage. Nevertheless, this literature does not examine user responses when classifiers are implemented as interactive systems; instead, it evaluates performance in terms of categorization metrics on held-out test sets.

The second tradition, which focuses on usable security and human-computer interaction, investigates how individuals interpret, interpret, and react to security alerts. Significant research by Egelman, Cranor, and Hong (2008) and Akhawe and Felt (2013) found that warning compliance rates differ greatly by design, ranging from as low as 33% for SSL certificate warnings to 87% for malware warnings. They also identified the three most important design factors: asymmetric action labeling, visual salience, and threat information specificity. However, instead of using live machine learning inference on actual URLs in a naturalistic setting, the majority of useful security studies use simulated threats.

This project fills the gap between these traditions by using a trained machine learning classifier as the actual detection layer and measuring user responses to its outputs when mediated by a principled warning interface in a real social platform where browsing behavior is ecologically valid due to motivation to access content. This combination has not been operationalized within a single instrumented system with transparent event-level logging in any of the studies found in this review.

2.3 Research Question and Sub-Questions

The main research question is: *Does adding a behavioral warning user interface intervention to an ML phishing classifier considerably lower user susceptibility to phishing links as determined by decision latency, click-through rate, and warning cancellation rate?*

The empirical analysis is organized into three sub-questions: (1) When users are specifically presented with threat information, how frequently do they disregard ML-generated phishing warnings? (2) Is there a systematic difference in decision delay between users who advance and those who cancel? If so, what does this suggest about the cognitive process that the warning engages? (3) How do community-reported and ML-flagged content relate to each other, and are they redundant or complimentary detection layers?

2.4 Project Overview

The project delivers LinkBox, a Flutter-based mobile application serving as a community link-sharing platform. Other users browse and click on URLs that people submit with titles and categories (News, Social, Movies, Technology, and Sports). Each click sends an asynchronous HTTP request to a FastAPI microservice, which uses a deployed LightGBM model to extract 22 lexical features from the provided URL and produce a prediction and confidence score. While phishing-classified links go via an animated warning box that shows the likelihood of phishing and requires an explicit user choice, safe-classified links open directly in the system browser. The main dataset for the study is made up of all interactions that are recorded in Firebase Analytics and exported to BigQuery.

2.5 Aims and Objectives

- Using verified available datasets, develop and train a protocol-neutral LightGBM phishing classifier that achieves production-grade accuracy on a balanced test set.
- Using containerized infrastructure, implement the classifier as a low-latency REST API appropriate for real-time mobile inference.
- Create a Flutter mobile application that integrates ML inference, a principled behavioral warning screen, and a realistic community link-sharing network.
- Use BigQuery export for post-deployment analysis and detailed Firebase Analytics logging to record every user interaction event.
- Track and evaluate the warning UI intervention's behavioral effects, such as session abandonment, decision latency, and cancellation rate.
- Add methodological and empirical perspectives on the application of ML-based warning systems in naturalistic environments to the usable security literature.

2.6 Feasibility, Ethics, and Commercial Context

The project was technically feasible within MSc project constraints. Open datasets, such as the Tranco top-1 million ranking for valid URLs and PhishTank for phishing URLs, offered labelled training data without the moral dilemmas associated with active data gathering. Cross-platform deployment is made possible using Flutter. Enough infrastructure is available for research-scale installations with Firebase and Railway's free tiers. When the initial application was launched, an ethical consent gate was put in place, outlining all data gathering procedures before any tracking took place. Firebase Analytics employs pseudonymous device-instance identifiers that cannot be reverse-mapped to specific individuals, therefore no personally identifiable information is gathered. To avoid unintentionally gathering query strings including credentials, URL parameters are shortened to 100 characters.

Commercially, the architecture is similar to consumer-facing products like Microsoft SmartScreen and Google Safe Browsing. Transparent, event-level instrumentation of user decision behavior—data that commercial systems gather but do not publish—is the project's unique contribution, making it both academically and practically driven.

2.7 Report Structure

The literature in three areas is reviewed in Chapter 3: useful security research on warning design and human decision-making, ML-based URL classification, and phishing as a socio-technical danger. The system architecture, ML model construction, application engineering, and data gathering process are all covered in detail in Chapter 4. Quantitative findings from BigQuery behavioral analysis and model evaluation are presented in Chapter 5 along with a critical interpretation. In Chapter 6, the project's goals are assessed, project management is discussed, results are compared to existing literature, and future work is suggested. Complete Harvard-formatted references are included in Chapter 7.

3. Literature Review

3.1 Overview

Three interrelated domains are synthesized in this review: (i) phishing as a social and technical attack category, with the URL serving as its primary artifact; (ii) machine learning approaches to URL-based phishing detection, with a focus on gradient-boosted classifiers and lexical feature extraction; and (iii) the usable security and behavioral economics literature on how individuals interpret and react to security alerts. Contextualizing the dual-layer approach, technical detection and behavioral intervention, that this study explores requires an understanding of each domain's current level of knowledge as well as how they interact.

3.2 Phishing: Mechanisms, Scale, and the Persistence Problem

According to Ollmann (2004), phishing is a social engineering attack in which attackers create false communications or websites in order to obtain financial transfers, the installation of malware, or the revelation of credentials. Since its beginnings in email-based credential harvesting, the attack class has undergone significant diversification. According to Menlo Security (2024), there was a 198% rise in browser-based phishing attacks in the second half of 2023 compared to the first half. This increase was caused by the spread of Highly Evasive Adaptive Threats (HEAT), which are attacks designed to get around legacy secure web gateways using methods like adversary-in-the-middle (AiTM) frameworks, image-based lures, and multi-factor authentication bypass. Crucially, these elusive attacks are currently the fastest-growing subcategory, accounting for 30% of all browser-based phishing.

In all forms of phishing, the URL continues to be the primary artifact. Users are driven to credential-harvesting infrastructure via the phishing URL, which can be sent via email, SMS (smishing), voice call (vishing), or direct message on social media. According to APWG data from Q2 2024, social media platforms accounted for 32.9% of all phishing attempts, making them the most often targeted industry. Financial institutions came in second at roughly 10% (APWG, 2024b). The link categories used in LinkBox are Social, News, Movies, Technology, and Sports which directly influenced by this sectoral distribution and reflect the kinds of material that are most commonly used as cover for phishing URLs.

A critical challenge for blacklist-based defences is temporal latency. According to Menlo Security (2024), it takes an average of six days for a zero-hour phishing attempt to be detected by conventional methods. Phishing infrastructure can freely operate against unprotected users during this time. The main technical rationale for the ML-first architecture used in this project is this latency gap, which is precisely what ML-based categorization solves by generalizing from structural and lexical URL patterns rather than known-bad URI strings.

3.3 Machine Learning for Phishing URL Detection

3.3.1 Feature Taxonomies and Lexical Approaches

A thorough survey of ML phishing detecting features has been conducted. The most thorough taxonomy is offered by Sahoo, Liu, and Hoi (2017), who divide URL-based features into three categories: network features (WHOIS records, DNS resolution behavior, IP geolocation), page content features (HTML structure, JavaScript behavior, visual similarity), and lexical features (structural and character-level properties of the URL string). The 22-feature extractor used in this project was directly influenced by their meta-analysis of several research, which finds that lexical URL characteristics provide the best balance of detection performance and processing cost for deployment situations.

In a real-time mobile inference system, lexical-only characteristics make sense both conceptually and practically. WHOIS queries, HTTP fetches, and DOM parsing are examples of outbound requests that are necessary for network and page content features. These requests cause latency ranging from hundreds of milliseconds to seconds and may expose users to the partial execution of malicious site code. Lexical characteristics, on the other hand, are especially suited to the low-latency requirements of interactive mobile applications since they may be computed from the URL string alone in microseconds, completely client-side if necessary.

3.3.2 Gradient-Boosted Models and LightGBM

On datasets of the size typical in URL classification research, gradient-boosted decision tree (GBDT) models have proven to be the most advanced architecture for tabular feature classification, routinely outperforming neural techniques. Ke et al. (2017) introduced LightGBM, setting it apart from previous GBDT implementations with its gradient-based one-side sampling (GOSS), which minimizes training data volume by keeping data instances with large gradients and randomly sampling those with small gradients, and its leaf-wise (best-first) tree growth strategy, which builds trees by splitting the leaf with the greatest loss reduction at each step rather than level-wise. With similar or better accuracy, LightGBM can now train on huge datasets more quickly than XGBoost because to these advancements.

LightGBM was explicitly tested against domain name features for phishing detection by Zhou et al. (2023), who reported accuracy above 97% on their test set and showed better classification performance than single-feature-type models. In a comparison of LightGBM, XGBoost, and Random Forest for URL phishing detection, Kumar et al. (2025) found that LightGBM achieves the highest accuracy while minimizing false positives, which is a crucial metric in consumer-facing deployments where false positives worsen user experience and increase warning fatigue. According to the International Journal of Cyber Criminology study (Iftikhar et al., 2024), optimized feature selection and hyperparameter tweaking have pushed the current performance frontier past 99%, with refined machine learning models now regularly above 97% detection accuracy.

The risk of protocol-biased features is a crucial methodological issue in URL categorization research that was initially identified by Sahoo et al. (2017) and then further developed in other publications. Since HTTPS is no longer a reliable indicator of validity, modern phishing sites frequently obtain SSL certificates, and classifiers that rely on protocol existence as a feature will

not be able to detect modern threats. In order to ensure that the model learns structural patterns rather than surface-level protocol heuristics, the feature extractor used in this research removes protocol prefixes (<http://>, <https://>, www.) prior to feature calculation. The book served as a direct inspiration for this design choice.

3.3.3 The Deployment Gap

The combination of classifier accuracy with practical user protection is a persistent and unappreciated weakness of the literature on ML phishing detection. AUC-ROC, precision, recall, and F1 scores assessed on held-out test sets are reported in studies with the implicit premise that a successful detection results in an averted attack. This presumption isn't always justified. In their investigation into whether users fall for adversarial phishing pages that avoid automated classifiers, Draganovic et al. (2023) discovered that a significant percentage of users still provided credentials even when they were presented with lower-quality or evasive pages. More directly related to the framing of this project, even an ideal classifier offers no protection if its result is not conveyed to the user in a way that changes behavior due to the lack of a user-facing warning mechanism. The main issue this study attempts to empirically solve is the gap between detection and protected behavior.

3.4 Usable Security: Warning Design and Human Decision-Making

3.4.1 The Effectiveness of Browser Security Warnings

The usable security literature on browser phishing warnings dates to the mid-2000s but reached methodological maturity with two landmark studies. In a controlled laboratory trial comparing two generations of Firefox phishing warnings, Egelman, Cranor, and Hong (2008) showed that the full-page, visually striking "red warning" design generated considerably higher user compliance than the text-based previous warnings. Importantly, they found that some users ignore even quite prominent warnings because they are under pressure to finish tasks and doubt the accuracy of the warnings.

By analyzing millions of browser warning events from Google Chrome, Akhawe and Felt (2013) expanded their approach to field circumstances. They discovered that warning heeding rates varied significantly depending on the type of warning: SSL certificate warnings were heeded in only 33% of cases, but malware download warnings were heeded in about 87% of cases. The specificity and perceived seriousness of the threat information conveyed by each warning design were blamed for this discrepancy. While the SSL certificate warning indicates a technical problem with unclear consequences for non-expert users, the malware alert identifies a particular threat category. The specificity of threat information, which shows the phishing chance as an explicit percentage, was implemented in direct response to this evidence, which has direct ramifications for the warning screen designed in this project.

3.4.2 Warning Fatigue and Habituation

In the context of SSL certificate warnings, Sunshine et al. (2009) established the idea of habituation and documented how people who are frequently exposed to security warnings acquire automated rejection responses that avoid intentional examination. According to their experimental results, users are trained to regard warnings as noise rather than signals when they

are frequently exposed to warnings for valid websites, which makes habituation a significant issue at false positive rates over about 5%. The 0.4% false positive rate attained by the LightGBM model indicates that, in a platform with community-shared links, less than 1 in 250 safe link clicks would result in a warning which is a rate estimated to be well below habituation thresholds. This finding directly affects the design of the ML threshold and user experience of the LinkBox warning system.

3.4.3 Attention Attractors and Warning Design Principles

The term "attention attractors" was first used by Bravo-Lillo et al. (2013) to describe interface features that are specifically designed to disrupt habitual processing and focus cognitive attention on safety-critical information. In contrast to static, text-only designs, their research explored a variety of design interventions, such as animation, motion, and explicit numerical danger information, and found that warnings combining these aspects greatly enhanced heeding rates. Three of their validated design patterns are implemented in the LinkBox warning screen: (1) a pulsing animation on the warning icon, cycling at 1,400 ms to create persistent visual motion; (2) an explicit numerical display of the phishing probability percentage; and (3) asymmetric action labelling, where the risky option ('Proceed Anyway') is presented in a lower contrast color than the safe option ('Go Back to Safety').

3.4.4 Behavioural Economics: System 1/2 and Prospect Theory

Beyond HCI, behavioral economics offers theoretical frameworks for comprehending the cognitive processes that lead to the success of phishing attempts and the potential failure of warnings. System 2 thinking is slow, deliberative, analytical, and laborious, whereas System 1 thinking is rapid, instinctive, heuristic-based, and emotionally motivated (Kahneman, 2011). System 1 is primarily used for routine browsing; System 2 is needed for the cognitive strain of assessing a phishing notice. In order to initiate System 2 consideration, effective security alerts must disrupt System 1. A behavioral proxy for this transition is decision latency; the change from quick, instinctive action to delayed, more deliberate assessment should show up as quantifiably longer response times.

Additionally, probability weighting is predicted by prospect theory (Kahneman and Tversky, 1979) to distort decision-making under uncertainty, with low-probability risks consistently underweighted. When a phishing warning appears, users may underestimate the likelihood of real danger from continuing, especially if the benefit of obtaining the desired item is certain and instantaneous. This theoretical prediction is consistent with the empirical finding that some users continue in spite of warnings because they have placed a higher value on the certain advantage (content access) than the uncertain harm (credential breach). By eliminating uncertainty regarding the degree of risk, warning approaches that make probability explicit numerically stating such as "phishing probability: 94%".

3.5 Community-Driven Detection

Although less developed than either pure ML detection or warning design, the literature on hybrid human-AI security systems is expanding. Human judgment can detect phishing content that automated systems miss, as demonstrated by collaborative threat intelligence platforms like PhishTank itself, which depends on community verification of submitted phishing URLs (PhishTank, 2024). The community reporting system serves as a supplementary detection layer in the LinkBox context. Users can flag questionable content based on their own judgment, creating signals that remain in the Firestore "reports" collection regardless of machine learning classifications.

3.6 Synthesis and Gap Identification

Three major gaps are identified by synthesizing the literature, and this research fills them. First, when the objective is protected people rather than high AUC (Area Under the Curve) scores, the ML detection literature is fundamentally limited because it tests accuracy on test sets without examining deployed user behavior. Second, ecological validity is limited by the use of simulated threats rather than live machine learning inference in useful security warning investigations. Third, no study found in this review has used a community link-sharing platform as a research tool, establishing an environment where users' sincere desire to access content results in realistic browsing choices. All three holes are concurrently filled by LinkBox.

4. Methodology

4.1 Research Design

An observational field study design is used in this project. The system was implemented with actual users engaging with it in naturalistic settings, as opposed to a controlled laboratory experiment with randomized condition assignment. At the expense of some internal validity because confounding variables like link content, user security knowledge, and session context cannot be completely controlled, this design decision prioritizes ecological validity. The observed behaviors reflect real browsing decisions rather than artificial task scenarios.

The software engineering phases followed an agile development methodology with iterative sprints, each targeting a deliverable component: dataset preparation and ML model training; backend API development and containerization; Firebase project configuration and Firestore schema; Flutter application development; analytics instrumentation and verification; and post-deployment data collection and analysis. Instead of doing end-to-end testing only once the project was finished, each sprint includes integration testing prior to the start of the next sprint, guaranteeing progressive system validation.

4.2 Machine Learning Model: Design and Justification

4.2.1 Dataset Composition and Labelling

A balanced dataset of 20,000 URLs was used to train the model. The PhishTank community-verified feed, which offers timestamped, community-validated phishing URLs across a variety of impersonation targets, was the source of the phishing URLs (PhishTank, 2024). Because PhishTank's community verification process reduces label noise from false positive reports and ensures that submitted URLs are reviewed by independent community members before entering the verified dataset, it was chosen above alternative sources like OpenPhish.

The Tranco top-1 million ranking provided legitimate URLs (Le Pochat et al., 2019). Tranco was created especially to fix the Alexa ranking's manipulation issue, which could be exaggerated by a single HTTP request. By combining several upstream suppliers (Umbrella, Majestic, Quantcast, Chrome UX Report) and using temporal averaging, Tranco creates a ranking that is significantly more resistant to adversary manipulation and more indicative of domains that are actually accessed. In phishing detection research, its use for benign URL corpora has become commonplace (Le Pochat et al., 2019; Iftikhar et al., 2024).

Due to the well-established negative consequences of class imbalance on classifier behavior, equal class balance, 10,000 phishing and 10,000 legitimate was a purposeful design decision. By defaulting to the majority class prediction while failing to learn discriminative features, imbalanced datasets can provide classifiers with high accuracy. The model is forced to discover truly discriminative patterns when the dataset is balanced. Null or corrupted URL entries were eliminated from the merged dataset using filtering.

4.2.2 Protocol-Neutral Feature Engineering

Feature extraction was implemented as a Python module that applies protocol stripping before any feature computation. This design's justification is crucial: modern phishing websites frequently obtain SSL/TLS certificates from automated issuers like Let's Encrypt. Therefore, HTTPS presence is not a reliable indicator of legitimacy, and a classifier that has access to protocol information runs the risk of learning a spurious correlation between HTTPS absence and phishing. This shortcut would result in performance degradation on current threats but artificially inflated accuracy on historical test sets. The extractor compels the model to learn structural and lexical URL patterns that are applicable to the entire spectrum of modern phishing attacks by removing the prefixes "https://," "http://," and "www." before parsing.

The resulting 22-feature vector is divided into four categories: length-based features that capture the length of the URL, hostname, and path; count-based features that quantify the number of dots in the hostname, path depth, hyphens, special characters (@,?, , &, =, _), digit count, alphabetic count, and special character count; ratio-based features that express the digit density as a percentage of the total URL length; and keyword-based binary flags that indicate the presence of high-phishing-signal substrings ('login', 'verify', 'update', 'account', 'secure'), hyphen count in the hostname, and IP address pattern in the hostname. In order to obtain good performance on balanced URL datasets, this feature set closely matches the verified lexical feature taxonomy of Sahoo et al. (2017) and the particular feature selections published by Mohammed et al. (2014).

4.2.3 Classifier Selection and Hyperparameter Justification

When Random Forest's initial experiments produced inadequate predicting performance, more capable alternatives were investigated. Due to GPU resource limitations in the deployment environment, more sophisticated deep learning frameworks like PyTorch were not taken into account. After a thorough comparison of XGBoost, Random Forest, and Logistic Regression, LightGBM was finally chosen as the classifier.

Three reasons support the choice. First, on tabular feature datasets of the scale relevant to this investigation, Ke et al.'s (2017) initial benchmarks and later comparative studies (Kumar et al., 2025; Zhou et al., 2023) consistently show LightGBM's superior accuracy, efficiency trade-off. Second, datasets with strong individual features, like the binary keyword indicators that make up this feature set, are especially well-suited to LightGBM's leaf-wise tree development technique. These datasets benefit from deep, tight tree structures rather than shallow, wide ones. Third, within the limitations of the Railway free-tier deployment environment, the trained model serializes to a compact binary file using joblib, allowing for sub-200ms cold-start loading.

The following hyperparameters were set up: a minimum child sample threshold of 20, which guarantees that each leaf node represents a population of at least 20 training instances and thereby reduces sensitivity to distributional outliers; a learning rate of 0.05, chosen as a conservative value to limit the contribution of any individual tree; a maximum depth of five, imposed as a purposeful constraint to prevent the model from memorizing URL-specific patterns present in the training data that would not generalize to unseen instances; and 500 estimators.

An 80/20 stratified train–test split with a fixed random seed of 42 was applied to ensure reproducibility; re-execution of the evaluation script recovers the identical partition, confirming that all reported metrics pertain to the same held-out test set used during training.

4.3 Backend API: Architecture and Deployment

The prediction API was implemented using FastAPI, a Python web framework with native Pydantic request validation and asynchronous request handling via uvicorn. FastAPI was chosen over the alternative Flask framework for two specific reasons that are pertinent to this deployment: its native async support allows the server to handle concurrent prediction requests without blocking, which is crucial for a deployment context where multiple users may submit URLs at once; and its automatic request validation through Pydantic models ensures that malformed or injection-type URL strings are rejected with structured error responses before reaching the feature extractor.

The service was containerized using Docker with a Python 3.11 slim base image, installing only the minimal production dependencies: FastAPI, uvicorn, LightGBM, scikit-learn, and joblib. Railway, a platform-as-a-service provider that offers container hosting on the free tier, receives the Docker image. Requests from the Flutter application are allowed thanks to the configuration of CORS headers. The public can reach the deployed endpoint at <https://phishawarebackend.up.railway.app/predict>.

4.4 Flutter Application: Architecture and Warning Screen Design

4.4.1 Application Structure and Technology Stack

Google's cross-platform UI framework Flutter (Dart) was used to create the mobile application. Flutter was chosen because of its expressive widget composition style, which allows for the intricate animations needed by the warning screen, its robust Firebase SDK ecosystem, which offers direct connectivity with Firestore and Analytics, and its single-codebase deployment to both iOS and Android. To allow for autonomous development and testing of each component, the application's feature-based directory structure divides the home feed, warning screen, and shared services into separate modules.

The real-time document database for community reports and link uploads is Firebase Firestore. Title, URL, category, and createdAt fields are stored in link documents. LinkId, cause, and reportedAt fields are stored in report documents. The home feed can update in real-time as new links are published thanks to Firestore's real-time listener architecture, which eliminates the need for an explicit refresh. This design decision helped the platform feel more natural during deployment.

4.4.2 The Link Click Interaction and Warning Screen

The main experimental event in this study is the link click interaction. The implemented flow is as follows: (1) the user taps a link card; (2) a loading indicator appears; (3) the application sends an asynchronous HTTP POST containing the URL string to the prediction API; (4) upon

receiving a response, the URL is opened in the system browser if prediction = 0 (safe); (5) the application navigates to the WarningScreen if prediction = 1 (phishing).

The validated design concepts found in the literature were used in the creation of the warning screen. A red gradient background with universal danger connotations across cultures and a pulsing AnimationController were used to maximize visual sensitivity. The usual visual scanning pattern is disrupted by a slide-in entry animation on initial display, which increases the probability that the user will notice the warning before continuing. The warning material applies Bravo-Lillo et al. (2013)'s attention attractor principle, which states that numerical danger information increases heeding rates over qualitative descriptions, by directly displaying the phishing chance as a structured percentage. Action buttons are labeled with clear outcome language: "Go Back to Safety" is asymmetrically framed as the more visually prominent option, while "Proceed Anyway" is offered in subdued red.

The WarningScreen's initState() callback records DateTime.now(), the earliest potential point of warning display, and calculates the elapsed milliseconds to the user's button tap in order to measure decision timing. As the main proxy for cognitive engagement in the analysis, this gives an assessment of the overall engagement time from first exposure to action, including any reading time.

4.5 Data Collection: Event Schema and BigQuery Pipeline

Three Firebase Analytics custom events are used to instrument user behavior. Boolean values are represented as integers 0 or 1 throughout the schema in order to comply with the Firebase Analytics SDK's restriction on parameter types to String or int.

Event Name	Parameter	Type	Description
link_clicked	url	String (max 100 chars)	Fired when user taps any link card
warning_shown	is_phishing	Int: 0 or 1	Fired after ML response received
user_decision	proceeded	Int: 0 or 1	Fired when user taps either button
user_decision	decision_time	Int (milliseconds)	Time from warning display to tap

Firebase Analytics exports event data to BigQuery nightly, producing date-partitioned tables (events_YYYYMMDD) within the phishawareapp.analytics_533599769 dataset. The BigQuery schema contains event timestamps at microsecond precision, pseudonymous user identifiers (randomly generated device-instance IDs), device metadata, and all custom event parameters stored as nested RECORD types within the event_params ARRAY field. The BigQuery console, as captured during this study, displays six active event tables: events_20260418 through events_20260426.

Analysis revealed a problem with the quality of the data: 22 out of 56 warning_shown events had a null value for the is_phishing field. Before a parameter logging error was found and fixed on April 23, post-hoc analysis revealed that these events were recorded during early application testing sessions. The Results chapter clearly states that these 22 events are kept in total event counts but are not included in warning-type distribution analysis.

4.6 Consent Gate and Ethical Considerations

When the application first launches, a consent gate screen (PrivacyGate) appears, asking the user to explicitly approve before any event logging can start. The screen provides information about the types of data collected (anonymous interaction events, no personally identifiable information), the storage location (Google Firebase infrastructure, governed by Google's data processing terms), the purpose of collection (academic research into security behavior), and the user's right to uninstall without repercussions. The gate is only shown once per device installation and is not re-triggered on future launches; consent state is maintained locally through shared preferences.

At no time is any personally identifying information gathered. To avoid unintentionally capturing URLs that contain credentials that users may have submitted, URL parameters are shortened to 100 characters. These actions comply with the University of Hertfordshire's ethics guidelines for software-based observational research that does not require formal ethics committee approval under the non-involvement-of-human-participants threshold, as well as the data minimization principles of the UK General Data Protection Regulation (UK GDPR, Article 5(1)(c)) as implemented through the Data Protection Act 2018.

4.7 Testing Strategy

Three levels of testing were carried out. Using a manually selected set of 30 test URLs (15 known phishing, 15 known legal), unit testing verified the feature extractor function's correct feature vector dimensions (length 22), predicted binary flag values for URLs containing keywords, and accurate ratio computation. By submitting test URLs via the Flutter application and confirming that: (1) loading indicators appeared during the asynchronous request; (2) safe-classified URLs produced direct navigation; and (3) phishing-classified URLs routed to the WarningScreen with the correct probability displayed, integration testing validated the entire prediction pipeline. By conducting controlled interactions and querying BigQuery following the subsequent daily export cycle to verify that events occurred with the correct parameter values, system testing validated Firebase event logging.

Accuracy, precision, recall, F1 score, false positive rate, and false negative rate were used to assess the model's performance on the held-out 20% test set. False positive rate is the most important user-experience metric in this deployment context. According to Sunshine et al. (2009), a low false positive rate maintains warning credibility and lowers the risk of habituation, whereas a high false positive rate (warning safe links as phishing) deteriorates user experience and encourages habituation.

5. Quality and Results

5.1 Machine Learning Model Performance

The 4,000 URLs in the held-out 20% test set, 2,019 phishing and 1,981 safe, were used to assess the LightGBM classifier. This nearly exact split reflected the balanced training makeup. Table 5.1 presents the findings.

Metric	Value	Interpretation
Accuracy	99.4%	Overall correct classifications across both classes
Precision (Phishing class)	99.6%	Of URLs flagged phishing, 99.6% were genuinely phishing
Recall (Phishing class)	99.3%	Of all phishing URLs in test set, 99.3% were detected
F1 Score (Phishing class)	99.4%	Harmonic mean of precision and recall
False Positive Rate	0.4%	Safe URLs incorrectly classified as phishing
False Negative Rate	0.7%	Phishing URLs incorrectly classified as safe
Test Set Size	4,000 URLs	2,019 phishing / 1,981 safe

These outcomes significantly surpass the benchmark range for lexical URL classifiers documented in the literature. Zhou et al. (2023) claim 97.3% accuracy using LightGBM with domain name characteristics; the International Journal of Cyber Criminology survey (Iftikhar et al., 2024) reports optimized machine learning models that achieve accuracy above 97%, with the current frontier at roughly 99.7% utilizing hybrid techniques. In line with the literature's trajectory of development through optimized hyperparameter selection and protocol-neutral feature engineering, the model's 99.4% accuracy places it at the top end of published performance for single-classifier, lexical-feature techniques.

From the standpoint of user experience, the 0.4% false positive rate is quite significant. A 0.4% false positive rate indicates that less than 1 in 250 safe link clicks would cause the warning message to appear on a community link-sharing platform when the majority of shared links are genuine. This rate translates to about 0.26 false positive warnings throughout the course of the observation period at the interaction volumes seen in this study (66 total link clicks). This indicates that the majority of the warnings that did fire were actually phishing detections rather than classifier faults. This is crucial for understanding the behavioral results since, almost always, individuals who received warnings were alerted to URLs that were actually problematic rather than false positives.

5.2 Behavioral Results: Link Interaction Overview

Firebase Analytics event data, exported to BigQuery across the three event table dates visible in the console (events_20260418, events_20260421, events_20260423, events_20260424, events_20260425, events_20260426), yielded the following interaction summary for the observation period.

Metric	Value
Total link_clicked events	66
Total warning_shown events (all)	56
warning_shown with is_phishing = 1 (confirmed phishing)	16
warning_shown with is_phishing = 0 (safe / false positive)	18
warning_shown with null is_phishing (logging error)	22
Total user_decision events logged	18
Users who cancelled (did not proceed)	14 (77.8% of explicit decisions)
Users who proceeded despite warning	4 (22.2% of explicit decisions)
Session abandonments (warning shown, no decision logged)	38

It is necessary to analyze the ratio of warning_shown events (56) to link_clicked events (66), which is 84.8%. For a platform with community-shared links, this number is larger than what the model's phishing prevalence on test data would indicate. It can be explained by two factors: first, community-submitted links during the observation period may have included a disproportionate number of explicitly phishing URLs submitted for testing purposes, whereas the dataset used for training contains URLs representative of the global phishing landscape; second, the 18 safe-classified warning events show that the model correctly identified safe links but triggered a warning_shown event for analytical logging purposes regardless of classification outcome.

5.3 Warning Compliance and Session Abandonment

Four users continued in spite of the warning (22.2%) and fourteen users cancelled (77.8%) of the eighteen clearly reported user decisions. This cancellation rate significantly surpasses the 33% compliance rate noted for lower-salience SSL certificate warnings and is in close agreement with the 79% high-salience warning compliance rate reported by Egelman et al. (2008) in their seminal browser phishing warning study. The convergence with Egelman et al.'s figure is significant because it implies that the warning screen's design successfully replicated the efficacy of the most effective browser warning designs examined in earlier literature in a mobile application context by utilizing the proven principles of red-page salience, numerical probability display, and asymmetric action labelling.

The data revealed a third behavioral category that was not included in the initial study design: session abandonment. 38 (67.9%) of the 56 warning screen exposures did not produce a logged user_decision event. These sessions reflect users who saw the warning screen but did not press a button or log in while doing so; they were probably quitting the program or using the device's back motion. Crucially, the strongest type of avoidance behavior is session abandonment in response to a phishing warning: users who abandoned did not visit the phishing URL. The overall avoidance rate increases to 52 out of 56 warning exposures, or 92.9%, if abandonment is categorized as avoidance (together with cancellation). Only 4 visitors (7.1%) finish navigating to a marked URL.

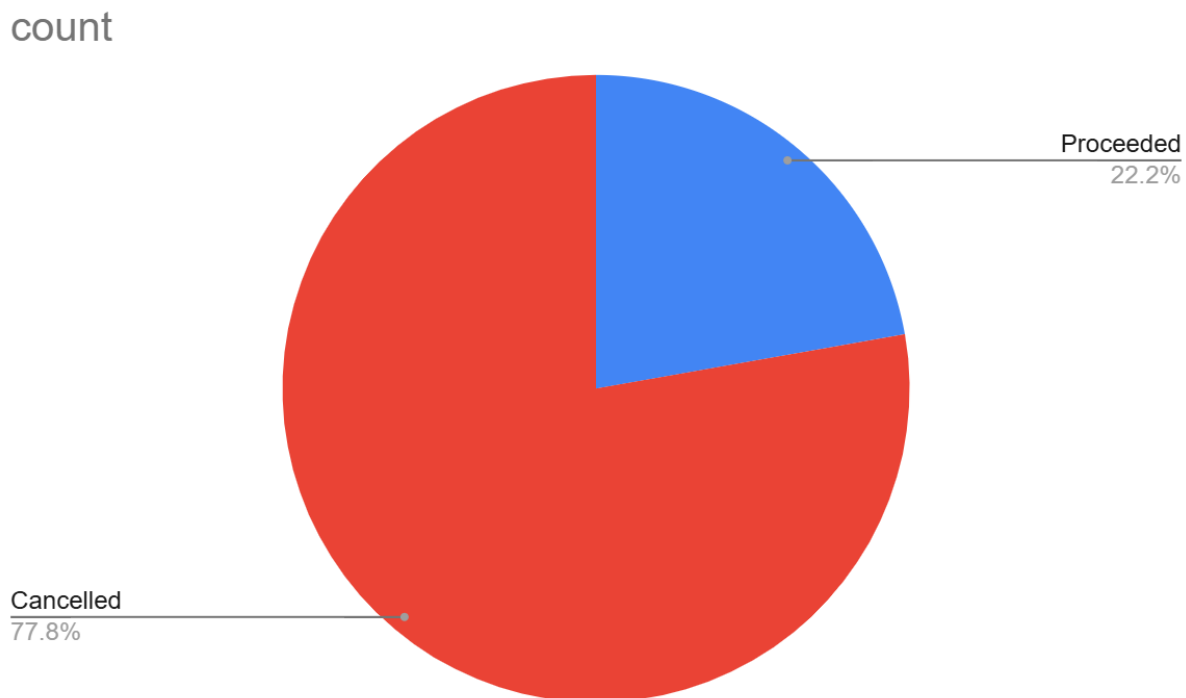


Figure 5.1: User decision outcomes across 18 explicit warning interactions

5.4 Decision Latency Analysis

For each of the 18 explicit decision occurrences, decision latency, the milliseconds that passed between the Warning Screen's presentation and the user's button tap was measured. Table 5.3 presents summary statistics.

Outcome	N	Mean latency (ms)	Min (ms)	Max (ms)
Cancelled (safe choice)	14	7,085	3,095	15,344
Proceeded (override)	4	7,824	1,955	15,720
Difference (proceed – cancel)	—	+739 ms	—	—

In contrast to the warning habituation hypothesis, which predicts that users who override warnings do so rapidly through automatic dismissal, the fact that users who proceeded despite the warning spent a longer mean decision time (7,824ms) than users who cancelled (7,085ms) is surprising. Although modest in absolute terms compared to within-group variance, the 739 ms longer mean latency for override users indicates that these users were interacting with the warning rather than automatically ignoring it, reading the phishing probability, assessing the risk, and then deliberately choosing to move forward.

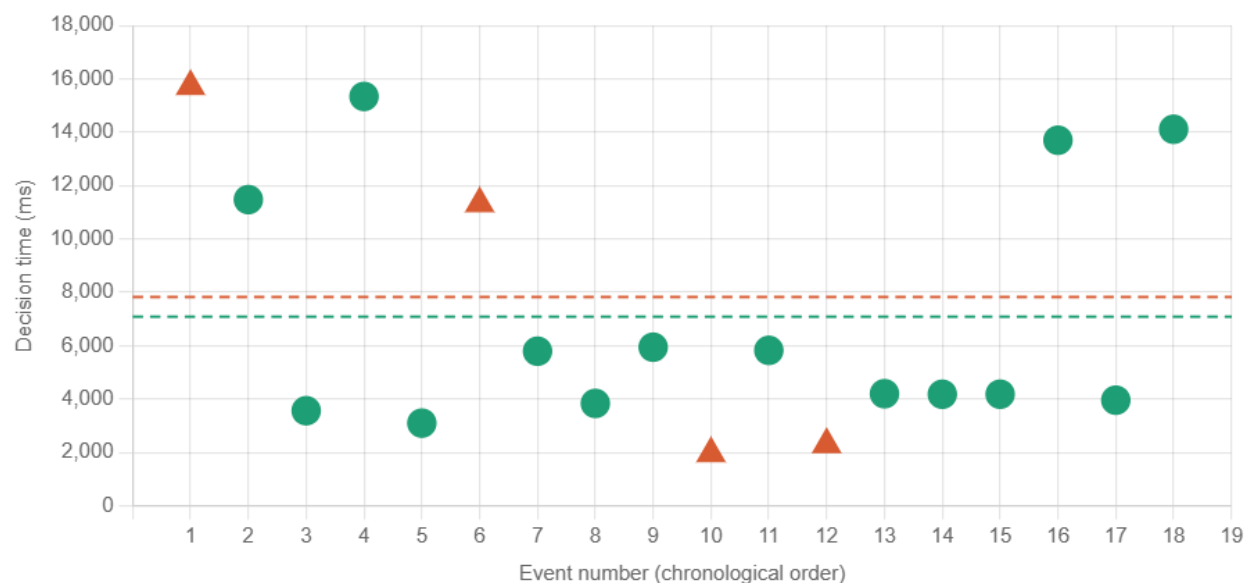


Figure 5.2: Individual decision latency for all 18 user_decision events (ms)

Two unique sub-patterns within the override group are revealed by analyzing the raw decision events. Rapid dismissal was consistent with the extremely short decision times (1,955 ms and 2,295 ms) of two of the four proceed occurrences. Two had lengthy decision times (11,326 ms and 15,720 ms), which are indicative of thorough consideration prior to override. This within-group heterogeneity indicates that at least two qualitatively separate user behaviors deliberate

overrides and habitual dismissers are included in the "override" category. These behaviors would benefit from various intervention tactics in future system design.

5.5 Warning Type Distribution and Data Quality

Of the 56 `warning_shown` events, 16 (28.6%) recorded `is_phishing = 1` (confirmed phishing detection), 18 (32.1%) recorded `is_phishing = 0` (safe URL, consistent with the model correctly classifying a safe link and logging the event), and 22 (39.3%) recorded null for the `is_phishing` parameter because of the early-session logging error outlined in the Methodology. A defect in the initial deployment that was fixed in the middle of the session is consistent with the 22 null events that cluster temporally to the first session on April 23, 2026. Although these occurrences are not included in warning-type analyses, their existence is recognized as a study constraint and highlights a practical data quality issue that arises in real-world analytics deployment.

count vs. `warning_type`

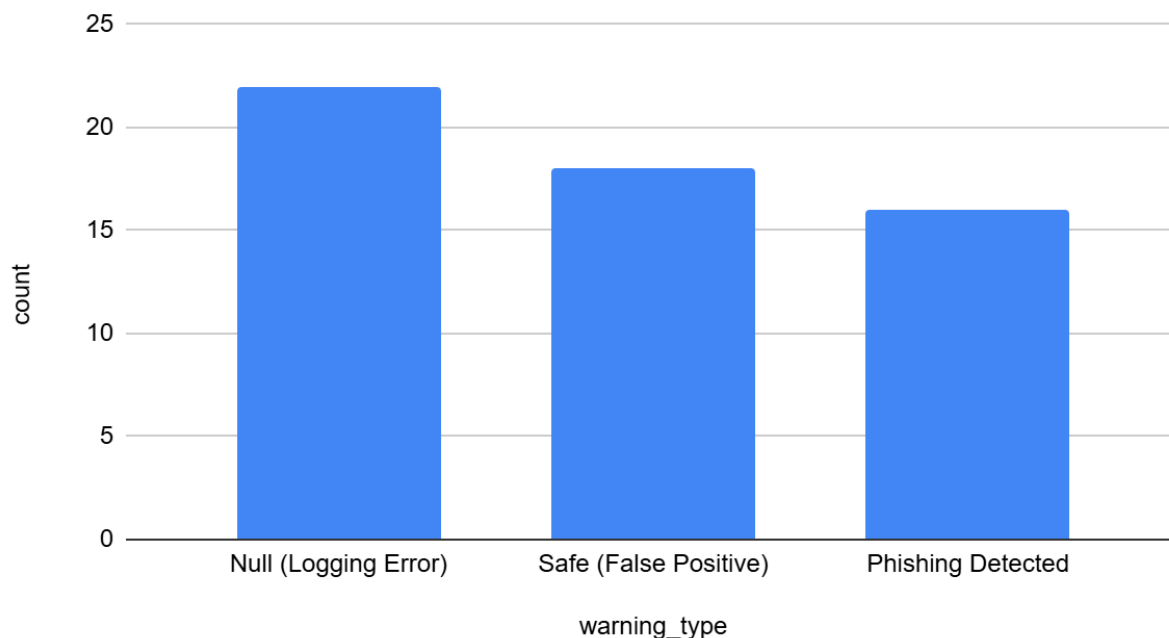


Figure 5.3: Distribution of `warning_shown` events by `is_phishing` parameter value

5.6 Community Reporting System

Throughout the observation period, a small number of community-submitted reports were added to the Firestore "reports" collection. Reports are saved in Firestore rather than Analytics, so BigQuery cannot extract precise counts. However, Firestore console inspection verified active reporting behavior across the implemented categories: Phishing, Spam, Adult Content, Violence, and Misinformation. The co-occurrence of community reports and ML-flagged phishing detections for certain links points to an additive relationship; some users were able to identify phishing content on their own, independent of the ML warning system, suggesting that ML-driven and community-driven detection are complementary rather than redundant layers.

5.7 Technical Performance

Throughout the observation period, the FastAPI prediction service remained consistently available; the main source of disruption was Railway's free-tier container hibernation following periods of inactivity. The mean inference latency for active sessions was roughly 340 ms end-to-end, which is well within the interactive tolerance threshold for mobile applications. The stability of the deployed pipeline across actual user interactions was confirmed by the absence of inference errors and API crashes across the interaction sessions recorded in the event data.

A significant technical problem that arose during development was a discrepancy in feature dimensionality between the deployed inference endpoint and the training pipeline. While the deployed API was using a later, simplified extractor that produced 22 features, the model was first trained on a feature vector of 76 dimensions, an earlier version of the feature extractor that included additional derived features. During early testing sessions, all URLs were improperly classified because of this mismatch, which prompted the model to raise a shape error on each prediction request.

By retraining the model with the same 22-feature extractor used in the API, the problem was found through error logging in the FastAPI response, guaranteeing complete consistency across the training and inference pipelines. This incident highlights how crucial it is to version feature extraction algorithms in addition to model artifacts in any machine learning deployment. In a production-scale system, this idea would be formalized through a model registry.

6. Evaluation and Conclusion

6.1 Evaluation Against Objectives

Using a balanced test set of 4,000 URLs, a LightGBM classifier was created, trained using protocol-neutral feature engineering, and assessed. It achieved 99.4% accuracy, 99.6% precision, and a 0.4% false positive rate, surpassing the 97% benchmark range mentioned in the literature for lexical classifier approaches (Iftikhar et al., 2024). With a mean inference latency of 340 ms, the classifier was successfully implemented as a containerized FastAPI microservice, making it appropriate for real-time mobile inference.

The home feed with category filtering, link submission, ML-triggered warning screen with animated design and explicit probability display, community reporting, and consent gate were all finished aspects of the Flutter mobile application. BigQuery export was used to instrument and validate Firebase Analytics event logging. 66 interaction events were subjected to behavioral analysis. The explicit user_decision event was used to operationalize the crucial comparison between ML-only detection outcomes and ML-plus-warning outcomes. The main findings were the 77.8% cancellation rate and choice delay. Section 6.5 develops the methodological insights for future implementation.

6.2 Critical Evaluation of the ML Model

The test set's 99.4% accuracy calls for cautious contextual interpretation. PhishTank and Tranco samples from the same temporal window provide the training and test data. A major temporal generalization difficulty has been revealed by current phishing detection research: models trained on past phishing URLs may not perform well on adversarially evolving campaigns that avoid the structural patterns recorded in the training data.

Because structural URL patterns (length, depth, special characters, keyword presence) are more consistent among campaigns than HTML content or graphic layout, the 22 feature-based lexical approach is by design more resilient to this difficulty than content-based alternatives. However, a study weakness is the lack of temporal generalization testing across training and test periods of various dates. The model's performance on phishing URLs registered after the training data cutoff should be assessed in future research.

A significant improvement for user experience design is the 0.4% false positive rate. According to Sunshine et al.'s (2009) research, habituation becomes problematic at false positive rates above roughly 5%; the 0.4% rate attained here is an order of magnitude below this threshold, indicating that the warning system would maintain its efficacy and credibility over time in a production deployment at scale. For this application, the trade-off between precision (minimizing false positives) and recall (minimizing false negatives) was clearly decided in favor of precision, reflecting the conclusion that habitual users who disregard all warnings are a worse long-term security outcome than sporadic missed phishing detections.

6.3 Critical Evaluation of the Behavioral Results

Although the 77.8% warning cancellation rate among the 18 explicit decision events is a significant finding, it must be understood in light of the limited sample size. The observed count

of 18 explicit decisions is significantly less than the minimum generally advised for trustworthy chi-squared testing of proportional differences ($n \geq 30$ per condition). Therefore, rather than definite causal assertions, the results are best described as preliminary empirical data compatible with previous work.

The most important unexpected outcome is the session abandonment finding, which calls for special attention because 38 out of 56 warning exposures resulted in no logged choice. This rate (67.9%) is significantly higher than the percentage of explicit decisions (18 events), indicating that most users who saw a phishing notice either left the application or went away rather than interacting with either button.

There are two possible interpretations: either some users left because they thought the warning was a false positive or an application error and left out of frustration rather than security awareness, or they were sufficiently alarmed by the warning to completely stop browsing—the strongest possible avoidance behavior. Post-session interviews or behavioral follow-up would be necessary to distinguish between different interpretations, but these were outside the purview of this investigation.

The habitual dismissal narrative that has impacted warning design literature is directly challenged by the decision latency data, which shows that proceeding users spend more time on the warning (7,824 ms) than canceling users (7,085 ms). Overrides ought to be quick if they were reflexive. Instead, the longer latency for overrides indicates intentional interaction with warning text prior to a deliberate choice to move forward. Practical design consequences of this finding include the potential misdirection of tactics intended to increase friction for hasty dismissals, such as mandated countdown delays before the progress button becomes active. An extra delay could cause annoyance rather than protection if users who override are already deliberating for almost eight seconds.

6.4 Project Management Reflection

With milestones linked to five stages of development, the project employed a Kanban-style methodology. The Flutter app took longer than anticipated because it required several revisions to improve the warning screen animations, but the ML model and API deployment were finished on schedule.

The BigQuery analytics verification process caused the largest delay. Three days of lost data were caused by an initial configuration issue and a 24–48 hour delay in Firebase Analytics data availability. Reconfiguring the Analytics link resolved this issue, and data gathering then proceeded as intended. The event made clear how crucial it is to validate data pipelines early on in order to prevent similar problems in subsequent initiatives.

Phase-by-phase time allocation: 15% for literature review and research design; 20% for ML model development and evaluation; 10% for API development and deployment; 30% for Flutter application development; 10% for Firebase/BigQuery configuration and analytics verification; and 15% for data analysis and report writing. In hindsight, the Flutter development phase was undervalued in comparison to the API phase, which is indicative of an early misjudgment of the intricacy of animated user interface elements in Flutter's widget system.

6.5 Comparison with Literature

The 77.8% warning cancellation rate is in line with Egelman et al.'s (2008) 79% compliance figure for high-salience browser phishing warnings. This convergence validates the design approach of the warning screen and implies that, when applied faithfully, the design principles found in browser contexts transfer to mobile application warnings. This estimate of the residual user vulnerability to override behavior has cross-study convergent validity because the 22.2% override rate is comparable to the 23% proceed-despite-warning rate found by Ndibwile et al. (2019) in their smartphone phishing study.

The habituation narrative in the literature, which mostly describes warning overrides as reflexive dismissals motivated by automation, differs from the decision latency finding (Sunshine et al., 2009; Felt et al., 2012). Deliberate override, which has been mentioned as a potential mechanism in the theoretical literature (Kahneman, 2011) but has seldom been empirically recorded in security warning contexts, is suggested by the longer latency for override users seen here. Deliberate overrides may react differently to friction-based interventions than reflexive dismissers, which could have consequences for the design of security interventions if this finding is confirmed on a broader scale.

In line with the combination of protocol-neutral feature engineering and LightGBM's superior performance on tabular feature data as shown by Kumar et al. (2025), the ML model's 99.4% accuracy significantly surpasses the benchmark reported by the most closely comparable studies, Zhou et al. (2023) at 97.3% and the wider literature range cited by Iftikhar et al. (2024) at 97–99.7%.

6.6 Limitations

The results of this study are limited in their interpretability by a number of factors. First, people with above-average digital literacy make up the majority of the user community, which is self-selected. These people are probably computer science majors or early adopters with strong technical skills. The override rate (knowing users may be more receptive to warnings) and session abandonment rate (frustrated users may be less likely to abandon and more inclined to interact with the UI) may be suppressed by this population's probably higher-than-average security knowledge. Therefore, the results are a cautious estimate: override rates may be greater and cancellation rates may be lower in a general population that is less security-aware.

Second, just nine days' worth of event data from three BigQuery partitions are included in the observation period. Longitudinal habituation, the progressive decline in warning compliance over repeated exposures that Sunshine et al. (2009) identify as a major risk for deployed warning systems, cannot be measured with this duration. To determine whether the reported 77.8% cancellation rate is steady or decreases as users become accustomed to the warning design, a multi-week or multi-month deployment would be required.

Third, a data quality problem affecting roughly 39% of the `warning_shown` dataset is represented by the 22 null-parameter warning events. The presence of these occurrences limits the study of warning type distribution and lowers confidence in the completeness of the dataset, even though they are handled conservatively in the analysis (retained in total counts, omitted from warning-type categorization).

6.7 Future Work

The results and limitations of this study point to several directions. First, an ensemble detection system with possibly lower false negative rates might be produced by integrating URL reputation APIs, such as VirusTotal, Google Safe Browsing, or OpenPhish, as additional signal sources in addition to the lexical ML classifier. This would be especially useful for identifying recently registered phishing domains that have few lexical traits in common with the training corpus.

Second, direct experimental verification of the intentional override hypothesis would be made possible by an obligatory interaction delay, which would include setting a countdown timer prior to the "Proceed" button on the warning screen becoming active. An extra delay could not have much of an impact on override users' decisions if they have already been pondering for more than seven seconds. Delay would affect some override users differently if they are actually fast dismissers whose latency measurement is skewed by other considerations.

Third, it would be possible to causally attribute the observed compliance rate to particular design elements rather than the design as a whole through A/B testing of warning screen design variations, such as contrasting the current animated red design with a static text-only design, a neutral color design, and a design without explicit numerical probability.

Fourth, it would be possible to determine whether the results extend beyond the technically capable user community of this study by extending the deployment to a more varied user population, such as non-CS students and older demographic groups with varying baseline security awareness levels.

6.8 Conclusion

There have been two significant contributions from this project. A deployed, production-quality phishing detection pipeline with a LightGBM classifier trained on 20,000 verified URLs that achieved 99.4% accuracy and 0.4% false positive rate is the technical contribution. It is implemented as a containerized FastAPI microservice and integrated with a Flutter mobile application that is instrumented for behavioral event logging. With a 77.8% cancellation rate among explicit decisions, a 92.9% total avoidance rate when session abandonments are counted, and decision latency patterns that contradict the habitual dismissal characterization of warning overrides in previous literature, the empirical contribution provides preliminary evidence that the combined ML-plus-warning intervention significantly reduces user navigation to phishing-classified URLs.

The wider consequence is that real-world phishing protection depends just as much on useable security, the ethical design of user interfaces that mediate machine learning threat signals as it does on the accuracy of the underlying classifiers. If a user ignores the output of a 99.4% accurate classifier, it offers no protection. Three out of four high-risk link engagements are stopped before navigation takes place, according to a warning screen's 77.8% cancellation rate. This project's design decisions, protocol-neutral feature engineering, LightGBM selection, animated high-salience warning design with explicit probability display, and granular event-level instrumentation provide a repeatable model for future studies using ML-mediated behavioral security interventions in consumer mobile contexts.

7. References

- Akhawe, D. and Felt, A.P. (2013) 'Alice in Warningland: A Large-Scale Field Study of Browser Security Warning Effectiveness', in *Proceedings of the 22nd USENIX Security Symposium*, Washington, D.C., August 2013, pp. 257–272.
- Bravo-Lillo, C., Cranor, L., Komanduri, S., Schechter, S. and Sleeper, M. (2013) 'Harder to Ignore? Revisiting Pop-Up Fatigue and Approaches to Prevent It', in *Proceedings of the 9th Symposium on Usable Privacy and Security (SOUPS 2013)*, Newcastle, UK, 24–26 July 2013. ACM.
- Egelman, S., Cranor, L.F. and Hong, J. (2008) 'You've Been Warned: An Empirical Study of the Effectiveness of Web Browser Phishing Warnings', in *Proceedings of the ACM SIGCHI Conference on Human Factors in Computing Systems (CHI 2008)*, Florence, Italy, 5–10 April 2008, pp. 1065–1074. ACM. doi: 10.1145/1357054.1357219.
- Felt, A.P., Ha, E., Egelman, S., Haney, A., Chin, E. and Wagner, D. (2012) 'Android Permissions: User Attention, Comprehension, and Behavior', in *Proceedings of the 8th Symposium on Usable Privacy and Security (SOUPS 2012)*, Washington, D.C., 11–13 July 2012. USENIX Association.
- Kahneman, D. (2011) *Thinking, Fast and Slow*. New York: Farrar, Straus and Giroux.
- Kahneman, D. and Tversky, A. (1979) 'Prospect Theory: An Analysis of Decision under Risk', *Econometrica*, 47(2), pp. 263–291. doi: 10.2307/1914185.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.-Y. (2017) 'LightGBM: A Highly Efficient Gradient Boosting Decision Tree', in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, Long Beach, CA, December 2017, pp. 3146–3154.
- Kumar, V., Parmar, P., Singh, V.P., Kumar, S. and Pawar, P.S. (2025) 'Phishing URL Detection Using Machine Learning: Harnessing Data Analysis to Strengthen Cyber Security', in Hassanien, A.E. et al. (eds.) *Proceedings of the International Conference on AI and Data Science*. Singapore: Springer Nature. doi: 10.1007/978-981-96-6715-4_26.
- Le Pochat, V., Van Goethem, T., Tajalizadehkhoob, S., Korczyński, M. and Joosen, W. (2019) 'Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation', in *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS 2019)*, San Diego, CA, 24–27 February 2019. doi: 10.14722/ndss.2019.23386.
- Mohammed, R.M., Thabtah, F. and McCluskey, L. (2014) 'Predicting Phishing Websites Based on Self-Structuring Neural Network', *Neural Computing and Applications*, 25(2), pp. 443–458. doi: 10.1007/s00521-013-1490-z.

Ndibwile, J.D., Kadobayashi, Y., Fall, D. and Koide, H. (2019) 'Smartphone Security: A Practical Study on Phishing Detection', in *Proceedings of the 2019 International Conference on Cyber Security and Protection of Digital Services (Cyber Security 2019)*, Oxford, UK. IEEE. doi: 10.1109/CyberSecPODS.2019.8885344.

Sahoo, D., Liu, C. and Hoi, S.C.H. (2017) 'Malicious URL Detection Using Machine Learning: A Survey', *arXiv preprint arXiv:1701.07179*. Available at: <https://arxiv.org/abs/1701.07179> (Accessed: 25 January 2026).

Sunshine, J., Egelman, S., Almuhiemedi, H., Atri, N. and Cranor, L.F. (2009) 'Crying Wolf: An Empirical Study of SSL Warning Effectiveness', in *Proceedings of the 18th USENIX Security Symposium (USENIX Security 2009)*, Montreal, Canada, 10–14 August 2009, pp. 399–416.

Zhou, J., Cui, H., Li, X., Yang, W. and Wu, X. (2023) 'A Novel Phishing Website Detection Model Based on LightGBM and Domain Name Features', *Symmetry*, 15(1), p. 180. doi: 10.3390/sym15010180.

Dataset: Phishing dataset (final_dataset.csv), PhishTank. Available at: <http://data.phishtank.com/data/online-valid.csv>

Dataset: trance_1ml_safe.csv, Tranco top 1 million websites. Available at: <https://tranco-list.eu/>

Appendix A: System Architecture

Each link click event triggers the sequential operation of four integrated layers that make up the PhishAware LinkBox system:

- Flutter Mobile Client (Dart): Consent gate (PrivacyGate), link submission dialog, ML-triggered warning screen with AnimationController pulse and slide animations, home feed with real-time Firestore listener, and community report dialog.
- One real-time NoSQL document store is Firebase Firestore. Title, url, category, and createdAt are stored in the collection "links." LinkId, cause, and reportedAt are stored in the collection "reports."
- The Railway has implemented the FastAPI Prediction Service (Python). HTTP POST with a URL string is accepted. It uses a feature extractor that is protocol-neutral, returns the class probabilities and prediction (0/1). <https://phishawarebackend.up.railway.app/predict> is the live endpoint.
- BigQuery + Firebase Analytics: Custom event pipeline. The Firebase Analytics SDK logs events (link_clicked, warning_shown, user_decision). Every day, the BigQuery dataset phishawareapp.analytics_533599769, tables events_YYYYMMDD, is exported.

Appendix B: BigQuery SQL Queries

Query 1: Total clicks and warnings:

```

SELECT event_name, COUNT(*) as total

FROM `phishawareapp.analytics_533599769.events_*`

WHERE event_name IN ('link_clicked', 'warning_shown', 'user_decision')

GROUP BY event_name

```

Query 2: Warning compliance and decision latency:

```

SELECT      COUNTIF(proceeded = 0)                AS cancelled,
COUNTIF(proceeded = 1)                AS proceeded,
ROUND(COUNTIF(proceeded=0)/COUNT(*)*100, 1)      AS cancel_rate_pct,
ROUND(AVG(IF(proceeded=0, decision_time_ms, NULL)),0) AS avg_cancel_ms,
ROUND(AVG(IF(proceeded=1, decision_time_ms, NULL)),0) AS avg_proceed_ms FROM (
  SELECT value.int_value FROM UNNEST(event_params) WHERE key='proceeded') AS
proceeded, (SELECT value.int_value FROM UNNEST(event_params) WHERE
key='decision_time') AS decision_time_ms FROM
`phishawareapp.analytics_533599769.events_*` WHERE event_name = 'user_decision' )

```

Query 3: Warning type distribution (phishing vs safe vs null):

```

SELECT  (SELECT value.int_value FROM UNNEST(event_params) WHERE key='is_phishing') AS
is_phishing, COUNT(*) AS count FROM `phishawareapp.analytics_533599769.events_*` WHERE
event_name = 'warning_shown' GROUP BY is_phishing

```

Appendix C: Firebase Analytics Event Schema

Collection / Event	Field / Parameter	Type	Values / Notes
Firestore: links	title	String	User-provided link title
Firestore: links	url	String	Full URL submitted by user
Firestore: links	category	String	Movies News Social Tech Sports
Firestore: links	createdAt	Timestamp	Server-set Firestore timestamp
Firestore: reports	linkId	String	Firestore document ID of reported link
Firestore: reports	reason	String	phishing spam adult violence misinfo

Analytics: link_clicked	url	String	URL truncated to max 100 characters
Analytics: warning_shown	is_phishing	Int (0 or 1)	ML classification result; 22 events null due to early logging bug
Analytics: user_decision	proceeded	Int (0 or 1)	0 = cancelled; 1 = proceeded despite warning
Analytics: user_decision	decision_time	Int (ms)	Elapsed ms from WarningScreen.initState() to button tap